Attendance Code: 790874



UNIVERSITY OF LINCOLN

CMP2028M

Natural Language Processing

Week 7: “Constituent and Dependency Parsing”

Heriberto Cuayahuitl

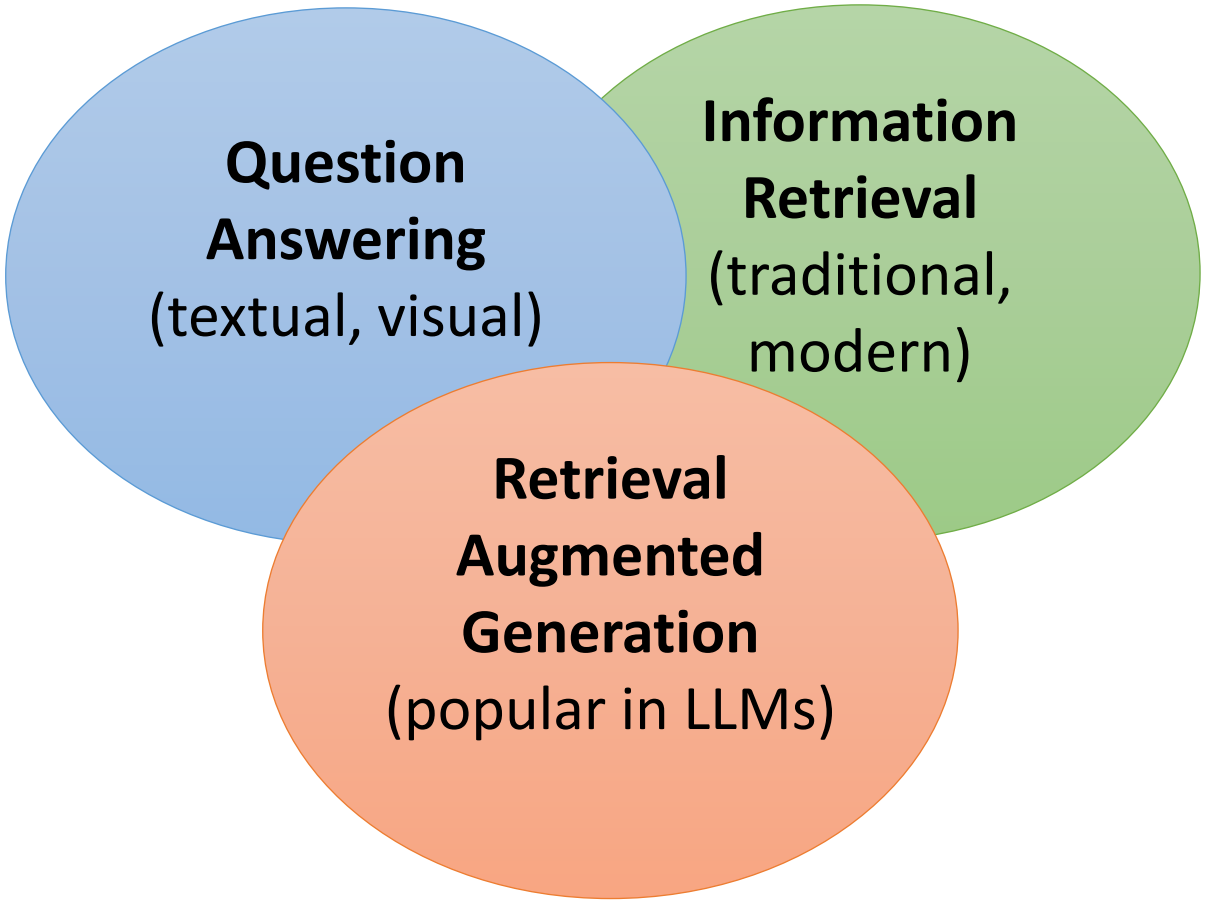
https://staff.Lincoln.ac.uk/hcuayahuitl

Agenda

- Week 1, 5-6 Feb: Tokenization and regular expressions
- Week 2, 12-13 Feb: N-gram language models
- Week 3, 19-20 Feb: Text classification
- Week 4, 27-27 Feb: Vector semantics and embeddings
- Week 5, 5-6 Mar: Large Language Models (LLMs)
- Week 6, 12-13 Mar: Question answering & information retrieval
- Week 7, 19-20 Mar: **Constituent and dependency parsing**
- Week 8, 26-27 Mar: Information extraction
- Week 9, 23-24 Apr: Overview of neural networks
- Week 10, 30 Apr-1May: Machine translation
- Week 11, 7-8 May: Chatbots and dialogue systems
- Week 12, 14-15 May: Speech recognition and synthesis

Easter break

Last Week


Question Answering
(textual, visual)
Information Retrieval
(traditional, modern)
Retrieval Augmented Generation
(popular in LLMs)

Today

- **Context Free Grammars (CFGs)**
- Constituent Parsing
- Dependency Parsing

Acknowledgement: Some images from https://web.stanford.edu/~jurafsky/slp3/18.pdf

Context-Free Grammars (CFGs)

A **Context-Free Grammar** (**CFG**) is a formal system for modelling structure in natural language.

A CFG consists of the following:

N	a set of **non-terminal symbols** (or **variables**)
Σ	a set of **terminal symbols** (disjoint from N)
R	a set of **rules** or productions, each of the form $A \rightarrow \beta$, where A is a non-terminal, β is a string of symbols from the infinite set of strings $(\Sigma \cup N)^*$
S	a designated **start symbol** and a member of N

Example CFG: Terminal Symbols

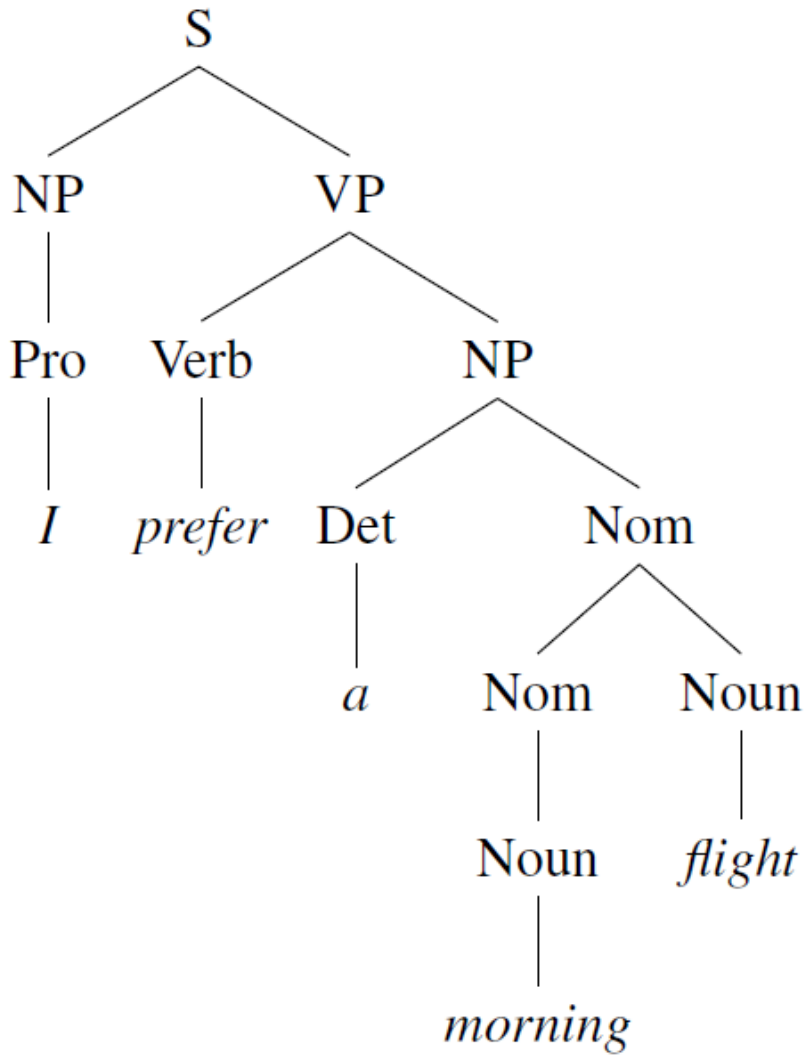
Noun →	*flights* | *flight* | *breeze* | *trip* | *morning*
Verb →	*is* | *prefer* | *like* | *need* | *want* | *fly* | *do*
Adjective →	*cheapest* | *non-stop* | *first* | *latest* | *other* | *direct*
Pronoun →	*me* | *I* | *you* | *it*
Proper-Noun →	*Alaska* | *Baltimore* | *Los Angeles* | *Chicago* | *United* | *American*
Determiner →	*the* | *a* | *an* | *this* | *these* | *that*
Preposition →	*from* | *to* | *on* | *near* | *in*
Conjunction →	*and* | *or* | *but*

Terminal symbols correspond to words in a language (a.k.a. 'the lexicon').

Example CFG: Rules, Non-Terminal Symbols, and Start Symbol

Grammar Rules			Examples
S	→	*NP VP*	I + want a morning flight
NP	→	*Pronoun*	I
	|	*Proper-Noun*	Los Angeles
	|	*Det Nominal*	a + flight
Nominal	→	*Nominal Noun*	morning + flight
	|	*Noun*	flights
VP	→	*Verb*	do
	|	*Verb NP*	want + a flight
	|	*Verb NP PP*	leave + Boston + in the morning
	|	*Verb PP*	leaving + on Thursday
PP	→	*Preposition NP*	from + Los Angeles

Example Parse Tree



The parse tree for “I prefer a morning flight” using the CFG in the previous two slides.

Two Uses of Contex-Free Grammars

A device for generating sentences.

Can be used to define and generate the set of grammatical sentences.

A device for assigning a structure to a given sentence.

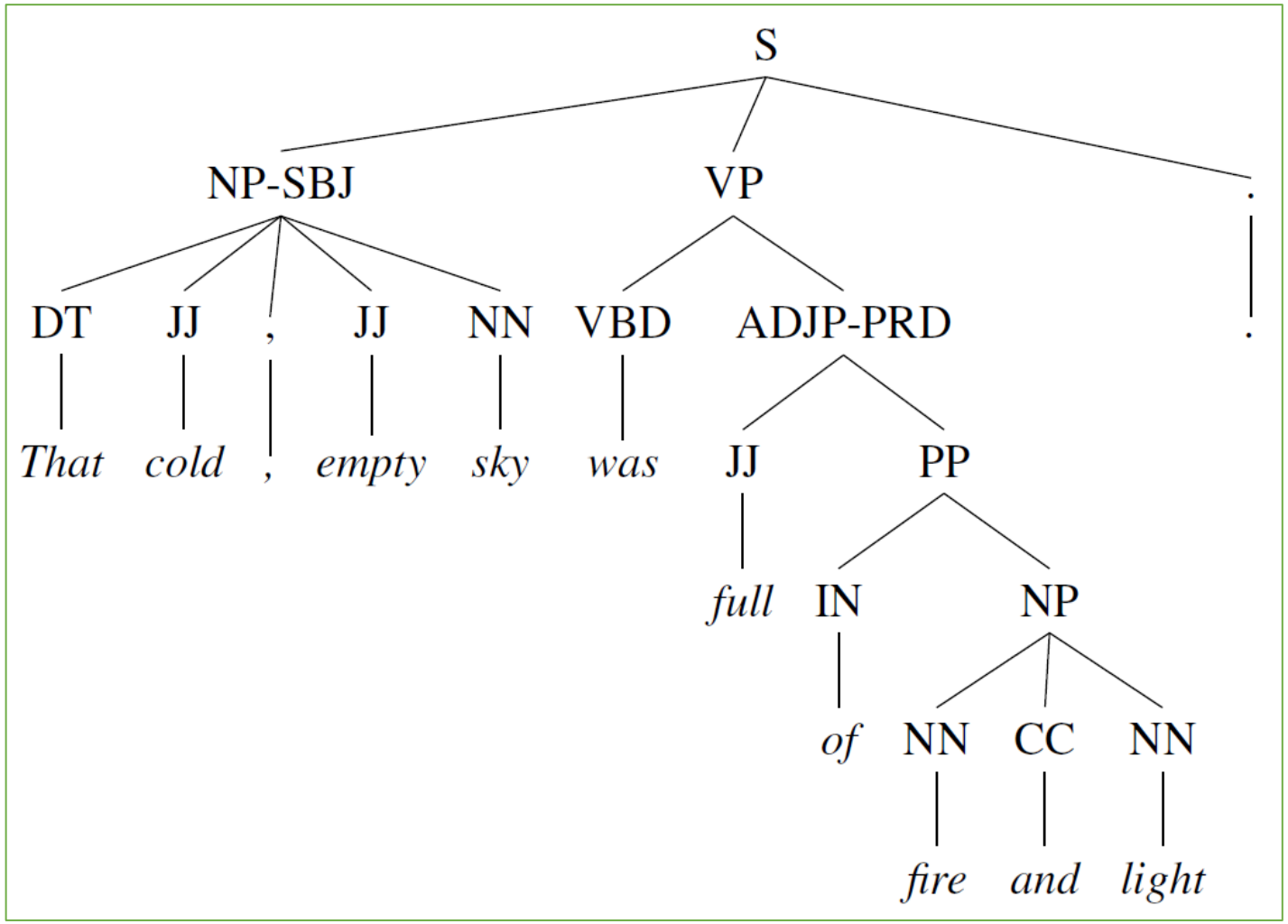
Can be used to analyse the syntactic structure of a given sentence (if it is grammatical).

Treebanks

A **treebank** is a corpus with sentences annonated with their corresponding parse trees.

```
((S
   (NP-SBJ (DT That)
     (JJ cold) (, ,)
     (JJ empty) (NN sky) )
   (VP (VBD was)
     (ADJP-PRD (JJ full)
       (PP (IN of)
         (NP (NN fire)
           (CC and)
           (NN light) ))))
   (. .) ))
```

LISP-style notation



Tree notation

Parse and parse tree for a Brown corpus sentence in the Treebank-3; they can be multi-lingual.

Today

- Context Free Grammars (CFGs)
- **Constituent Parsing**
- Dependency Parsing

Acknowledgement: Some images from https://web.stanford.edu/~jurafsky/slp3/18.pdf

CKY Parsing

Constituent parsing is used to analise a sentence via groups of words called constituents (phrases).

The **CKY algorithm** is widely used for constituent parsing, but it requires a grammar compatible with the **Chomsky Normal Form (CNF)**.

A CNF is restricted to rules of the form $A \rightarrow B\ C$ or $A \rightarrow\ w$; i.e., they have binary trees.

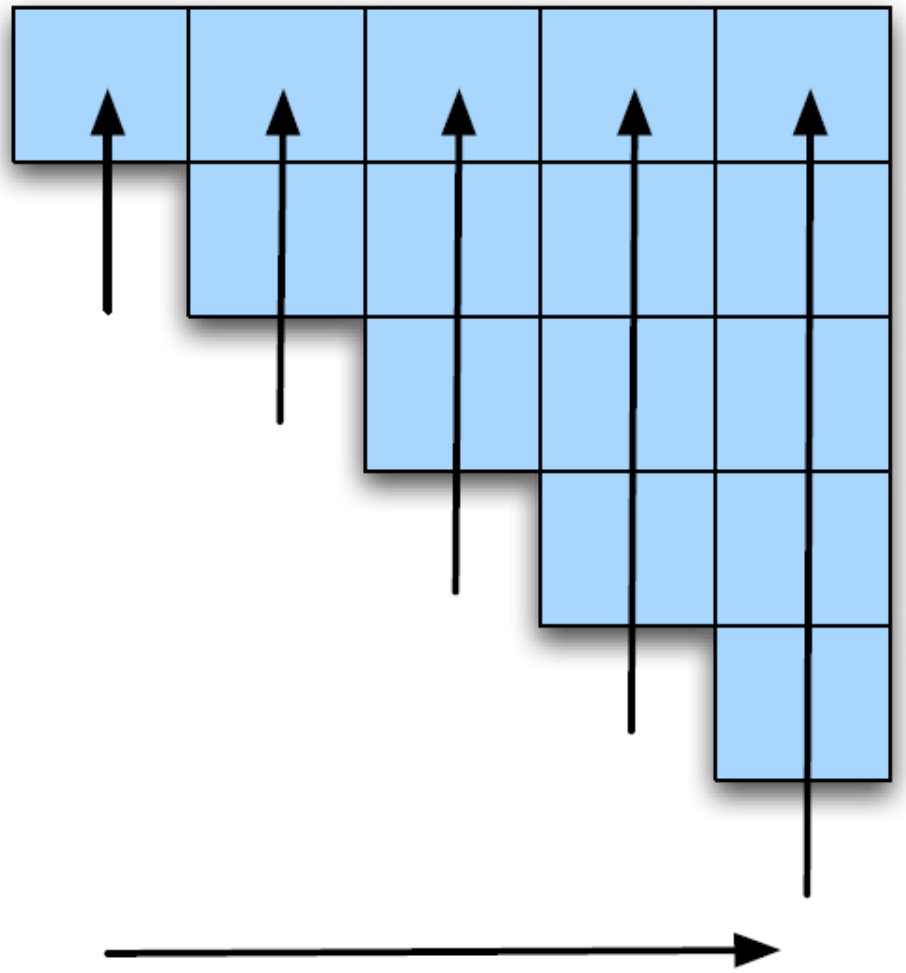
Since sentences can have multiple parses, CKY can be used to find all possible parse trees under a CFG.

CKY Parsing: Grammar Conversion

$\mathcal{L}_1$ Grammar	$\mathcal{L}_1$ in CNF
$S \rightarrow NP\ VP$	$S \rightarrow NP\ VP$
$S \rightarrow Aux\ NP\ VP$	$S \rightarrow X1\ VP$
	$X1 \rightarrow Aux\ NP$
$S \rightarrow VP$	$S \rightarrow book \mid include \mid prefer$
	$S \rightarrow Verb\ NP$
	$S \rightarrow X2\ PP$
	$S \rightarrow Verb\ PP$
	$S \rightarrow VP\ PP$
$NP \rightarrow Pronoun$	$NP \rightarrow I \mid she \mid me$
$NP \rightarrow Proper\text{-}Noun$	$NP \rightarrow United \mid Houston$
$NP \rightarrow Det\ Nominal$	$NP \rightarrow Det\ Nominal$
$Nominal \rightarrow Noun$	$Nominal \rightarrow book \mid flight \mid meal \mid money$
$Nominal \rightarrow Nominal\ Noun$	$Nominal \rightarrow Nominal\ Noun$
$Nominal \rightarrow Nominal\ PP$	$Nominal \rightarrow Nominal\ PP$
$VP \rightarrow Verb$	$VP \rightarrow book \mid include \mid prefer$
$VP \rightarrow Verb\ NP$	$VP \rightarrow Verb\ NP$
$VP \rightarrow Verb\ NP\ PP$	$VP \rightarrow X2\ PP$
	$X2 \rightarrow Verb\ NP$
$VP \rightarrow Verb\ PP$	$VP \rightarrow Verb\ PP$
$VP \rightarrow VP\ PP$	$VP \rightarrow VP\ PP$
$PP \rightarrow Preposition\ NP$	$PP \rightarrow Preposition\ NP$

CKY Parsing: Parse Table

Book	*the*	*flight*	*through*	*Houston*
S, VP, Verb Nominal, Noun [0,1]	[0,2]	S,VP,X2 [0,3]	[0,4]	S,VP,X2 [0,5]
	Det [1,2]	NP [1,3]	[1,4]	NP [1,5]
		Nominal, Noun [2,3]	[2,4]	Nominal [2,5]
			Prep [3,4]	PP [3,5]
				NP, Proper-Noun [4,5]



Parse table for “Book the flight through Houston”.

CKY Parsing: Explanation

- A 2D matrix is used to encode the structure of a parse tree.
- The table is indexed by (i, j), with i=start & j=end of a span.
- Each cell $[i, j]$ contains possible non-terminals that can generate that substring.
- Since each non-terminal symbol has two children, a constituent can be split into two parts (sub-constituents):
  - the first constituent $[i, k]$ must lie to the **left** of entry $[i, j]$ somewhere along row i, and
  - the second entry $[k, j]$ must lie **beneath** it, along column j.

CKY Parsing: Algorithm

function CKY-PARSE(*words, grammar*) **returns** *table*

for $j \leftarrow$ **from** 1 **to** LENGTH(*words*) **do**
  for all $\{A \mid A \rightarrow words[j] \in grammar\}$
    $table[j-1, j] \leftarrow table[j-1, j] \cup A$
  for $i \leftarrow$ **from** $j-2$ **down to** 0 **do**
    for $k \leftarrow i+1$ **to** $j-1$ **do**
      for all $\{A \mid A \rightarrow BC \in grammar$ **and** $B \in table[i,k]$ **and** $C \in table[k, j]\}$
        $table[i,j] \leftarrow table[i,j] \cup A$

Indexes:
i = start position of a substring/span
j = end position of a substring/span
k = split point inside the substring
`table[i=`start index`, j=`end index`]`

Whilst the first "for all" fills the diagonal (cells `table[j-1, j]`), the other "for all" fills the non-diagonal cells.

CKY Parsing: Algorithm (cont.)

function CKY-PARSE(*words, grammar*) **returns** *table*

for $j \leftarrow$ **from** 1 **to** LENGTH(*words*) **do**
  for all $\{A \mid A \rightarrow words[j] \in grammar\}$
    $table[j-1, j] \leftarrow table[j-1, j] \cup A$
  for $i \leftarrow$ **from** $j-2$ **down to** 0 **do**
    for $k \leftarrow i+1$ **to** $j-1$ **do**
      for all $\{A \mid A \rightarrow BC \in grammar$ **and** $B \in table[i,k]$ **and** $C \in table[k, j]\}$
        $table[i,j] \leftarrow table[i,j] \cup A$

For each span (i, j):
  Try every split point k
  Look at:
    left part: table[i, k]
    right part: table[k, j]
  If a rule $A \rightarrow BC$ matches:
    Add A to table[i, j]

For span (1,3):
  Try split point k=2
  Look at:
    left part: table[1, 2]=Det
    right part: table[2, 3]=Nominal,Noun
  Since $NP \rightarrow Det\ Nominal$ matches:
    Add NP to table[1, 3]

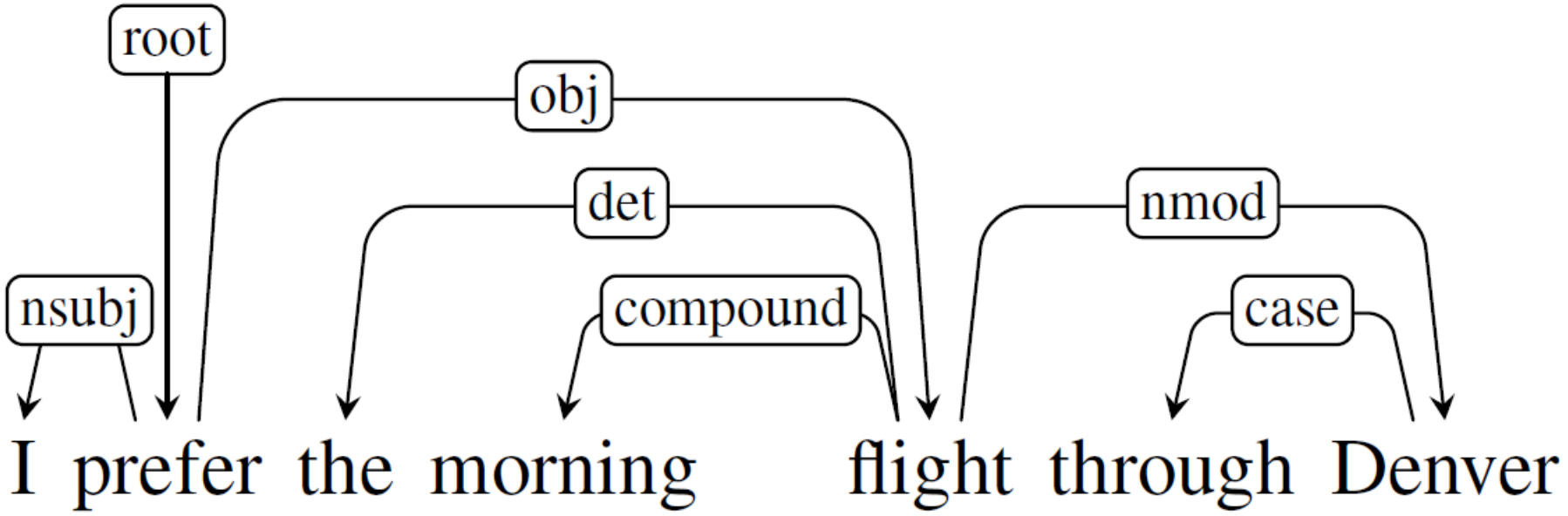
Today

- Context Free Grammars (CFGs)
- Constituent Parsing
- **Dependency Parsing**

Acknowledgement: Some images from https://web.stanford.edu/~jurafsky/slp3/19.pdf

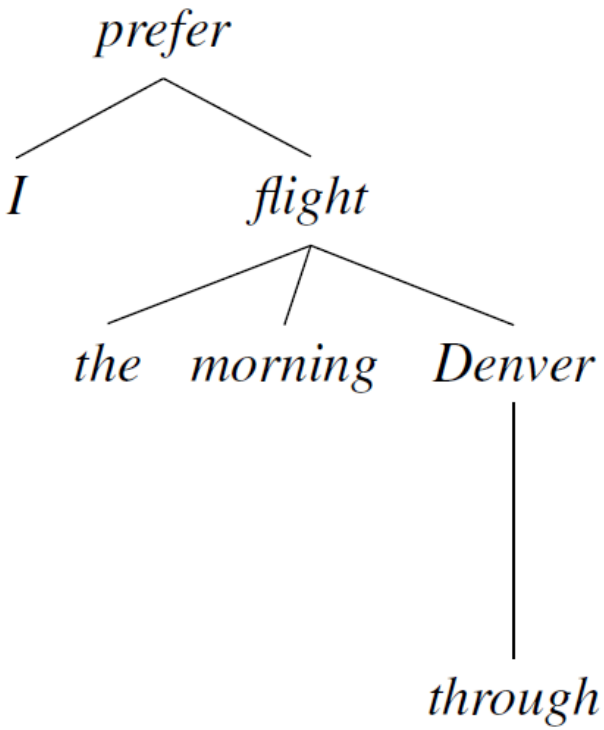
What is Dependency Parsing?

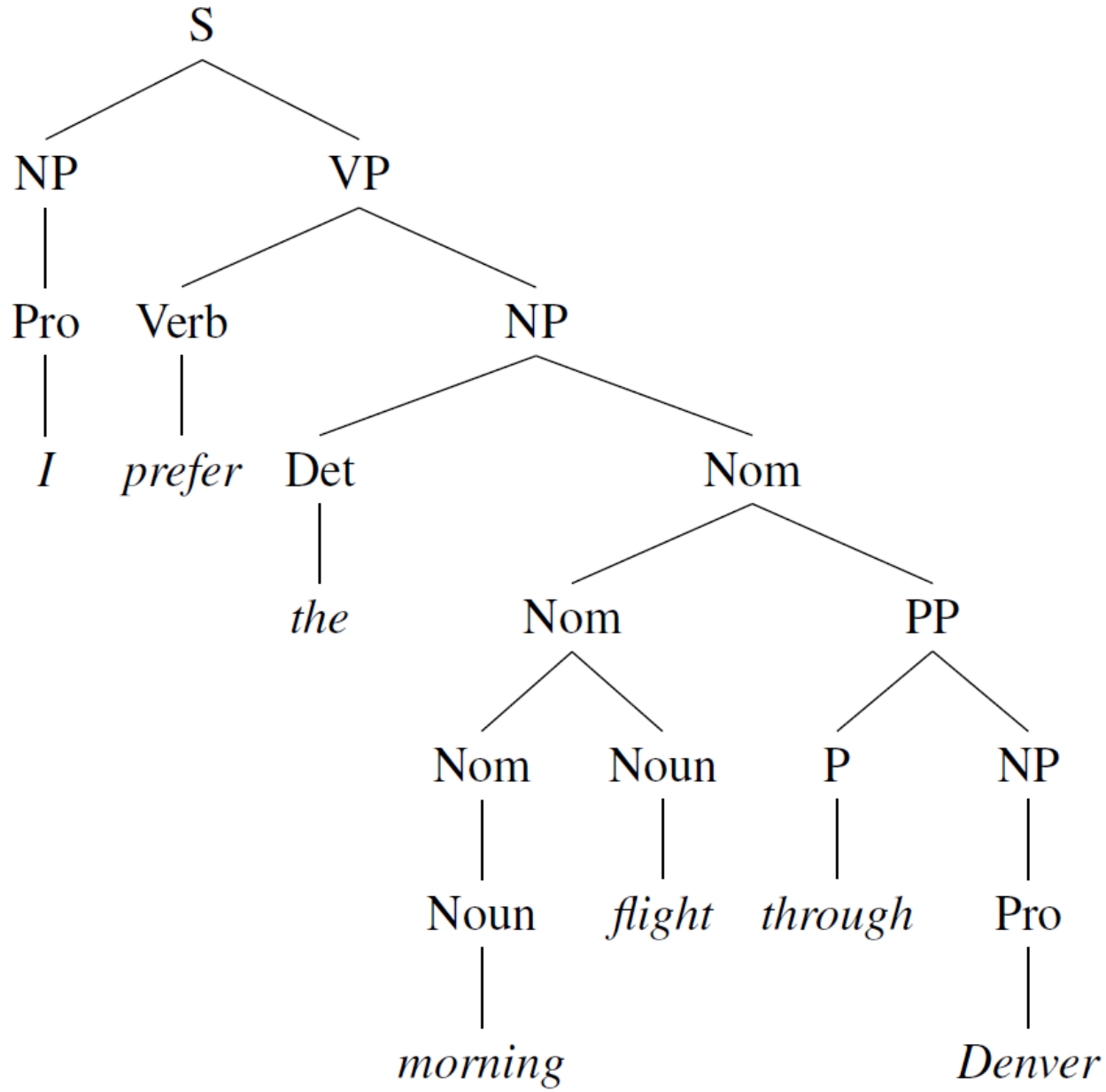
In **Dependency Parsing**, the syntactic structure of a sentence is described in terms of directed binary grammatical relations between the words, as we can see here:



The parse tree for “I prefer a morning flight” using the CFG in the previous two slides.

Dependency vs. Constituent Parsing





Dependency Parsing: Formalisation

A dependency structure can be represented as a **directed graph** $G = (V, A)$, consisting of vertices V (words or punctuation), and a set of pairs of vertices A called arcs.

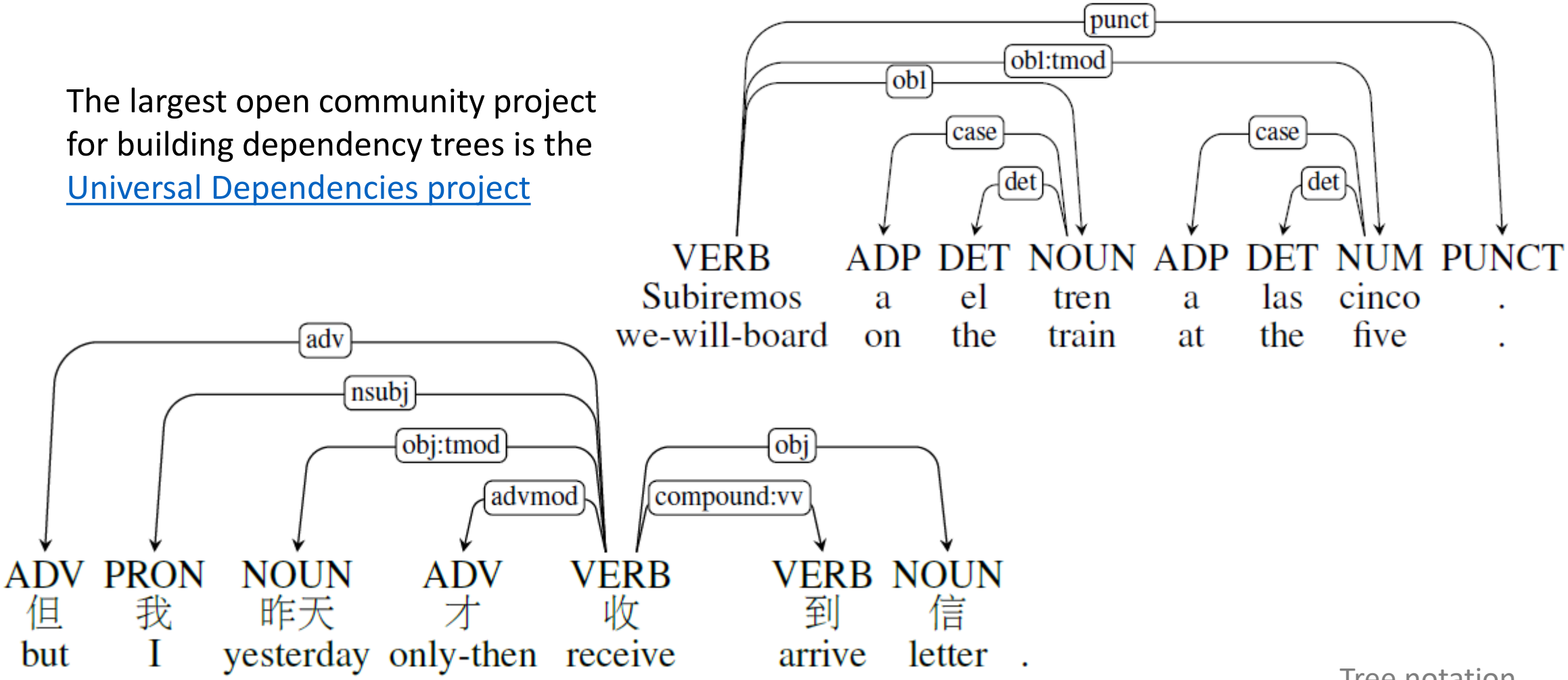
A dependency tree is a directed tree with these constraints:

- There is a single designated root node that has no incoming arcs.
- Except for the root node, each vertex has exactly one incoming arc.
- There is a unique path from the root node to each vertex in V.

Since sentences can have multiple parses, CKY can be used to find all possible parse trees under a CFG.

Dependency Treebanks

The largest open community project for building dependency trees is the [Universal Dependencies project]



Tree notation

Examples showing dependency trees for sentences in Spanish, Basque, and Mandarin Chinese

Evaluating Parsers

$$\textbf{labeled recall:} = \frac{\#\text{ of correct constituents in hypothesis parse of } s}{\#\text{ of total constituents in reference parse of } s}$$

$$\textbf{labeled precision:} = \frac{\#\text{ of correct constituents in hypothesis parse of } s}{\#\text{ of total constituents in hypothesis parse of } s}$$

$$F1 = \frac{2 * Precision * Recall}{Precision + Recall}$$

This requires a human-labeled reference parse tree for each sentence in the test set.

Parsing Applications

Constituent Parsing:

- Grammar checking (to suggest missing/incorrect constituents)
- Machine translation (to map the syntactic structure of a sentence)
- Information extraction (to identify constituents in a sentence)

Dependency Parsing:

- Relation extraction (to identify for example person-activity or symptom-disease relationships)
- Semanti role labelling (to determine who did what to whom)
- Sentiment analysis (to link opinions to their entities in text)
- Text classification (to extract dependency features $w_1 \rightarrow w_2$)

Among others.

Today

- **Context Free Grammars** (**CFGs**): A CFG is a formal set of rules that defines how words and phrases can be combined to form valid sentences in a language.

- **Constituent Parsing**: Analyses a sentence by breaking it into nested phrases and representing its hierarchical phrase structure as a tree—requires a grammar.

- **Dependency Parsing**: Identifies relationships between words in a sentence, producing a tree that shows how words relate to each other.

What's Next?

- Readings:
  - Jurafsky and Martin. Speech and Language Processing. Chapters 18,19, 2026. [recommended]
  - Mayo M. "Text Preprocessing and Feature Engineering with spaCy". Statology, blog, 2025.

- Workshop **tomorrow**: Syntactic parsing programs for text classification.

- Next lecture: Information Extraction